

Un approccio pratico a **Java Swing** e **MVC**

by Cicio Ionut

Sommario

Java Swing	4
Wireframe	4
Implementazione passo passo di un wireframe	4
Creare una finestra con <code>JFrame</code> (Java API doc)	4
Aggiungere <code>Component</code> a una finestra con <code>JPanel</code> (Java API doc)	5
Come vengono disegnati i componenti: <code>paint(Graphics g)</code>	5
Centrare un componente con <code>GridBagLayout</code> (Java API doc)	7
Implementazione completa	8
Interfacce con Swing senza variabili d'appoggio	9
Modificare font e colori con <code>UIManager</code> (Java API doc)	11
Chiavi di <code>UIManager</code>	11
Layout Manager	12
BorderLayout (Java API doc)	12
Barra delle statistiche e menu di gioco	13
Implementazione	14
Implementazione pt.2	16
Funzionamento del <code>BorderLayout</code>	18
CardLayout (Java API doc)	19
Menu, impostazioni e partita (<i>cambiare schermata</i>)	19
L'implementazione più rozza	20
Usando gli enum	20
Singleton + Observer	21
GridLayout (Java API doc)	23
Implementazione	23
GridBagLayout (Java API doc)	25
Il layout più flessibile	25
Implementazione	26
In conclusione (sui Layout)	29
Pulsanti e Functional Interfaces	30
Disegnare programmaticamente con le trasformazioni	31
Semplificare il codice con <code>.translate(int x, int y)</code>	31
Trasformazioni affini	32
Timer	34
Diversi tipi di <code>Timer</code> paragonati	34
Ritardare azioni	34
Animazioni	34
Framerate del gioco	34
In conclusione su Java Swing	35
MVC	36
Model	36
View	36
Controller	36
Minesweeper (<i>prato fiorito</i>)	37
UML e modello	37
Vincoli del modello	38
[<code>Constraint.Game.victory_condition</code>]	38
[<code>Constraint.Game.loss_condition</code>]	38
Operazioni sul modello	39
Wireframe	39
Codice	40
Model	40
Encapsulation con public final	41

Tipizzazione con <code>enum</code>	41
Valori indefiniti: <code>null</code> vs <code>Optional<T></code>	41
Le <code>switch</code> expression (<code>switch</code> con steroidi)	41
Gli assegnamenti come espressioni	41
Observer Pattern	41
Alcuni esempi di <code>Stream</code>	42
Gestione del timer	43
Il metodo <code>update(Observable o, Object arg)</code> e <code>instanceof</code> ..	44
Controller	45
View	46
In conclusione su MVC	46
Integrare il progetto con GitHub	47
Usare una project manager (<i>Maven</i>)	47
GitHub Pages	48
Releases	48
GitHub Actions	48
Anatomia di una GitHub Action	49
Generare e pubblicare la documentazione in automatico	50
Generare l'eseguibile in automatico	51
Tips & tricks	52
Compressione di immagini e audio	52
Cosa fare se lo <code>.zip</code> pesa troppo per la consegna	52
Compressione lossy vs loseless	52
Esempio di compressione	52
Compressione audio	53
LSP + Formattazione automatica del codice	54
Shortcut per gli editor	54

Java Swing

L'obiettivo di questo capitolo è fornire, tramite esempi pratici, gli **strumenti fondamentali** per lo sviluppo *agevole* di interfacce grafiche con Java Swing.

Wireframe

Per sviluppare l'interfaccia grafica, che si tratti di un sito web, un'applicazione, un gioco etc... è utile disegnare un wireframe. Un wireframe è un **diagramma** fatto di rettangoli, testo, icone e frecce (come in Figura 1) che permette di **progettare rapidamente** un'interfaccia intuitiva e funzionale, semplificando il lavoro da fare durante la fase d'implementazione.

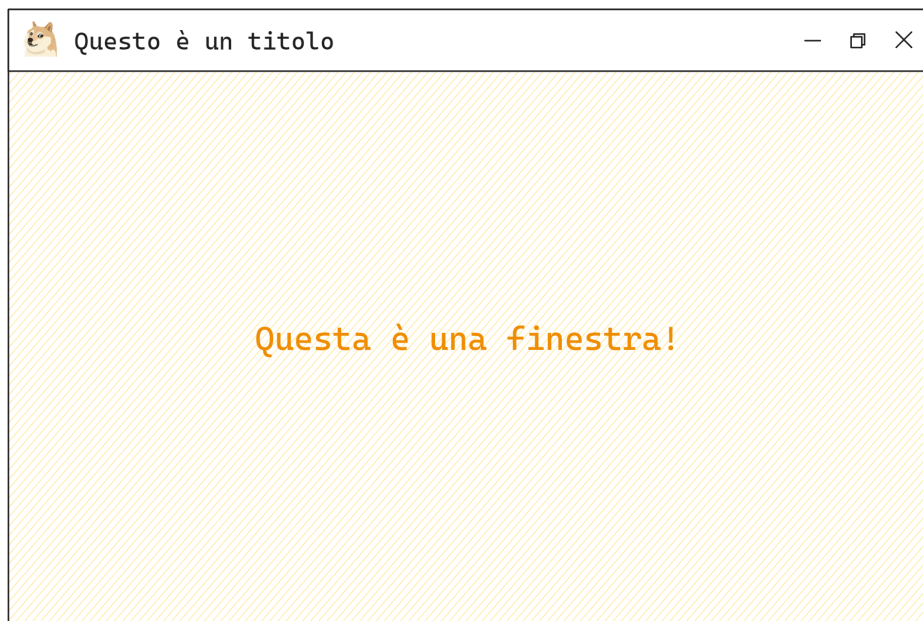


Figura 1: esempio di wireframe disegnato con [excalidraw](#)
di seguito ne viene discussa l'implementazione

Implementazione passo passo di un wireframe

Creare una finestra con `JFrame` ([Java API doc](#))

```
JFrame frame = ❶ new JFrame("Questo \u00E8 un titolo");
❷ frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

try {
    ❸ frame.setIconImage(ImageIO.read(new File("icon.png")));
} catch (IOException e) { }

❹ frame.setSize(600, 400);
❺ frame.setLocationRelativeTo(null);
❻ frame.setVisible(true);
```

Swing ha alcune impostazioni di **default particolari**:

- ❶ istanzia una finestra **invisibile**, per visualizzarla si usa ❹
- la finestra, alla chiusura, viene **solo nascosta**: per fare in modo che l'intera applicazione venga terminata si usa ❷
- ❸ posiziona la finestra al centro dello schermo

Per ridimensionare una finestra, oltre a ❹, si può usare anche con il metodo `pack()` che imposta le **dimensioni minime** per disegnare il contenuto della finestra.

ⓘ Nota

`pack()` imposta le **dimensioni a 0** se la finestra non ha contenuto, per cui bisogna invocare `pack()` **dopo** aver aggiunto componenti al `JFrame`.

Il prossimo passo è quello di aggiungere il testo "Questa è una finestra"

ⓘ Nota

Si ricordi di **cambiare sempre l'icona** ❸ di default :)

Aggiungere Component a una finestra con `JPanel` ([Java API doc](#))

```
JPanel panel = ❶ new JPanel();
JLabel label = ❷ new JLabel("Questa \u00E8 una finestra!");

panel.add(label);
frame.add(panel);
```

Sia ❶ sia ❷ sono `java.awt.Container` per cui dispongono del metodo `add(Component)`. Swing mette a disposizione vari `Component` già pronti:

- `JLabel` per **testo e immagini**
- `JButton` per **pulsanti**
- `JPanel` per interfacce più complesse
- `JTable`, `JSlider` etc... per utilizzi più specifici

ⓘ Nota

`java.awt.Container` è un esempio di [Composite Pattern](#)

Come vengono disegnati i componenti: `paint(Graphics g)`

Il passo successivo è quello di colorare lo sfondo e il testo come in Figura 1

```
JPanel panel = new JPanel() {
    @Override
    ❶ protected void paintComponent(Graphics g) {
        super.paintComponent(g);

        int density = 5;
        g.setColor(Color.decode("#ffec99"));
        for (int x = 0; x ≤ getWidth() + getHeight(); x += density)
            ❸ g.drawLine(x, 0, 0, x);
    }
};
```

```

    }
};
panel.setBackground(Color.WHITE);

JLabel label = new JLabel("Questa \u00E8 una finestra!");
label.setForeground(Color.decode("#f08c00"));
label.setFont(new Font("Cascadia Code", Font.PLAIN, 22));

panel.add(label);
frame.add(panel);

```

Il `JFrame` invoca il metodo `paint(Graphics g)` del `JPanel`. A sua volta `paint(Graphics g)` invoca in ordine:

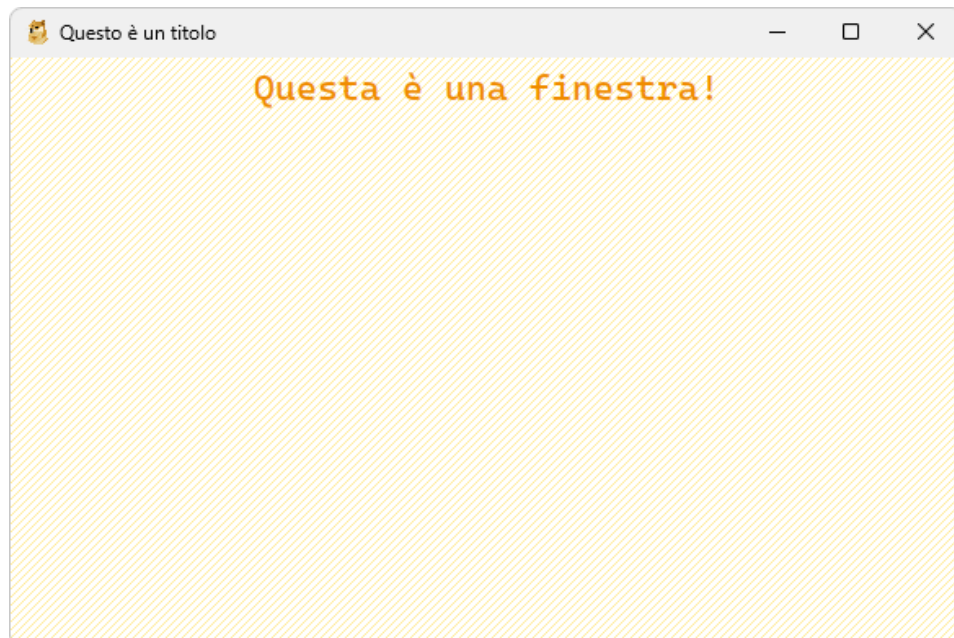
- `paintComponent(Graphics g)` ❶
- `paintBorder(Graphics g)`
- `paintChildren(Graphics g)`

Creando una **classe anonima** si possono sovrascrivere il comportamento di uno di questi metodi, in particolare quello di ❶ per disegnare lo sfondo con le linee oblique ❸ in Figura 1.

Questo approccio è molto flessibile, perché con `Graphics g` ❷ si possono disegnare immagini, testo e figure **programmaticamente** (quindi eventualmente **animazioni**).

ⓘ Nota

Il font `"Cascadia Code"` non è installato di default, si possono provare anche altri font



Centrare un componente con `GridBagLayout` ([Java API doc](#))

```
JPanel panel = ❶ new JPanel(new GridBagLayout());
```

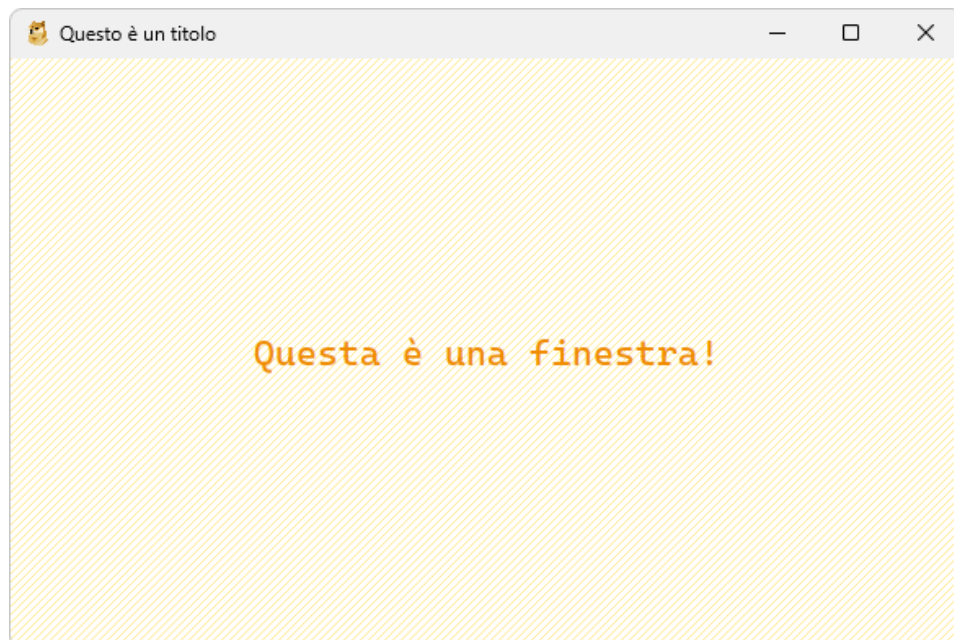
Il costruttore ❶ permette di specificare una **strategia** per **posizionare** e **dimensionare** il contenuto del `JPanel` in **automatico**:

- non bisogna calcolare a mano x, y, width e height dei componenti, lo fa il `LayoutManager` ❷ specificato
- il layout persiste anche quando la **finestra viene ridimensionata**

ⓘ Nota

`LayoutManager` è un esempio di [Strategy Pattern](#)

Per **centrare un elemento** in un panel si usa il `GridBagLayout` (la spiegazione è in seguito nella sezione sui `LayoutManager`)



Implementazione completa

Di seguito l'implementazione completa dell'interfaccia in Figura 1.

```
import javax.imageio.ImageIO;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;

import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;
import java.awt.GridBagLayout;
import java.io.File;
import java.io.IOException;

public class App {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Questo \u00E8 un titolo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        try {
            frame.setIconImage(ImageIO.read(new File("icon.png")));
        } catch (IOException e) {
        }

        JPanel panel = new JPanel(new GridBagLayout()) {
            @Override
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);

                int density = 5;
                g.setColor(Color.decode("#ffec99"));
                for (int x = 0; x <= getWidth() + getHeight(); x += density)
                    g.drawLine(x, 0, 0, x);
            }
        };
        panel.setBackground(Color.WHITE);

        JLabel label = new JLabel("Questa \u00E8 una finestra!");
        label.setForeground(Color.decode("#f08c00"));
        label.setFont(new Font("Cascadia Code", Font.PLAIN, 22));

        panel.add(label);
        frame.add(panel);

        frame.setSize(600, 400);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```


Interfacce con Swing senza variabili d'appoggio

Java mette a disposizione uno strumento che si chiama **instance initialization block**, un blocco di codice che viene eseguito dopo aver invocato il costruttore. È particolarmente comodo quando si istanziano **classi anonime**.

```
class Persona {
    String nome;
}

class Main {
    public static void main(String[] args) {
        Persona rossi = new Persona() {
            ❶ {
                nome = "Rossi";
            }
        };

        System.out.println(rossi.nome); /* Rossi */
    }
}
```

Si può sfruttare questa feature ❶ per riscrivere l'implementazione

```
public class App extends JFrame {
    App() {
        super("Questo \u00E8 un titolo");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        try { setIconImage(ImageIO.read(new File("icon.png"))); }
        catch (IOException e) { }

        add(new JPanel(new GridBagLayout()) {
            {
                setBackground(Color.WHITE);
                add(new JLabel("Questa \u00E8 una finestra!") {
                    {
                        setForeground(Color.decode("#f08c00"));
                        setFont(new Font("Cascadia Code", Font.PLAIN, 22));
                    }
                });
            }
        });

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            int density = 5; g.setColor(Color.decode("#ffec99"));
            for (int x = 0; x ≤ getWidth() + getHeight(); x += density)
                g.drawLine(x, 0, 0, x);
        }
    }

    setSize(600, 400);
}
```

```
        setLocationRelativeTo(null);  
        setVisible(true);  
    }  
  
    public static void main(String[] args) { new App(); }  
}
```

Modificare font e colori con UIManager [\(Java API doc\)](#)

L'aspetto dei componenti e il loro comportamento in Java Swing viene gestito con [javax.swing.LookAndFeel](#). È possibile modificare **globalmente** l'aspetto del [LookAndFeel](#) tramite [javax.swing.UIManager](#) (sostanzialmente un [HashMap](#) con diversi attributi per il tema).

```
UIManager.put("Label.font" ❶, new Font("Cascadia Code",  
Font.PLAIN, 14));
```

Ad esempio, con la chiave ❶ si può impostare il [Font](#) per tutti i [JLabel](#)

ⓘ Nota

[UIManager](#) è un esempio di [Singleton Pattern](#)

```
import java.awt.Color;  
import java.awt.Font;  
import javax.swing.UIManager;  
  
public class App {  
    static {  
        UIManager.put(  
            "Label.font",  
            new Font("Cascadia Code", Font.PLAIN, 14)  
        );  
        UIManager.put("Label.foreground", Color.DARK_GRAY);  
        UIManager.put("Label.background", Color.WHITE);  
  
        UIManager.put("Button.font", new Font("Cascadia Code",  
Font.PLAIN, 14));  
        UIManager.put("Button.foreground", new Color(224, 49, 49));  
        UIManager.put("Button.background", new Color(255, 201, 201));  
  
        UIManager.put("Button.highlight", Color.WHITE);  
        UIManager.put("Button.select", Color.WHITE);  
        UIManager.put("Button.focus", Color.WHITE);  
  
        UIManager.put("Panel.background", new Color(233, 236, 239));  
    }  
}
```

Chiavi di UIManager

È possibile generare l'elenco di chiavi disponibili, o provare [queste](#)

```
javax.swing.UIManager  
    .getDefaults()  
    .keys()  
    .asIterator()  
    .forEachRemaining(System.out::println);
```

Layout Manager

Il costruttore `JPanel(LayoutManager layout)` permette di specificare una **strategia** per **posizionare** e **dimensionare** il contenuto di un `JPanel` in **automatico**:

- non bisogna calcolare a mano x, y, width e height dei componenti, lo fa il `LayoutManager` specificato
- il layout persiste anche quando la **finestra viene ridimensionata**

◉ Nota

`LayoutManager` è un esempio di [Strategy Pattern](#)

`BorderLayout` ([Java API doc](#))

Si supponga di voler implementare questo wireframe



Figura 2: caso d'uso di un `BorderLayout`

C'è una barra con le statistiche in alto e lo spazio rimanente è occupato da un rettangolo centrale con un pulsante.

Barra delle statistiche e menu di gioco

```
public class App extends JFrame {
    static { UIManager.put("Panel.background", Color.WHITE); }

    App() {
        super("JGioco");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        try { setIconImage(ImageIO.read(new File("icon.png"))); }
        catch (IOException e) { }

        add(new JPanel(new BorderLayout(10, 10) ❶) { // Sfondo
            {
                ❷ setBorder(
                    BorderFactory.createEmptyBorder(20, 20, 20, 20));

                add(new JPanel() { // Barra in alto
                    {
                        ❸ setBackground(new Color(255, 255, 255, 0));
                        setBorder(BorderFactory.createCompoundBorder(
                            BorderFactory.createLineBorder(Color.decode("#2f9e44")),
                            BorderFactory.createEmptyBorder(10, 10, 10, 10)));
                    }
                }, BorderLayout.NORTH ❹);

                add(new JPanel() { // Blocco centrale
                    {
                        ❸ setBackground(new Color(255, 255, 255, 0));
                        setBorder(BorderFactory.createCompoundBorder(
                            BorderFactory.createLineBorder(Color.decode("#2f9e44")),
                            BorderFactory.createEmptyBorder(10, 10, 10, 10)));
                    }
                }, BorderLayout.CENTER ❺);
            }
        });

        @Override
        protected void paintComponent(Graphics g) { // Sfondo linee
            oblique verdi
            super.paintComponent(g);
            int d = 5; g.setColor(Color.decode("#b2f2bb") ❻);
            for (int x = 0; x ≤ getWidth() + getHeight(); x += d)
                g.drawLine(x, 0, 0, x);
        }

        setSize(600, 400);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public static void main(String[] args) { new App(); }
}
```

Breve analisi del codice:

- ❶ crea un `BorderLayout` specificando lo spazio da lasciare fra i componenti del `JPanel` (10px in verticale e 10px in orizzontale).
- ❷ aggiunge un bordo trasparente al `JPanel` (in modo che i componenti non siano attaccati al bordo della finestra).
- ❸ rende lo sfondo trasparente, perché il quarto parametro del costruttore di `Color` indica il **canale alpha**, ovvero la trasparenza del colore.
- ❹ e ❺ sono delle costanti che il `BorderLayout` usa per capire dove posizionare il componente (sono spiegate nella sezione sul funzionamento del `BorderLayout`).

ⓘ Nota

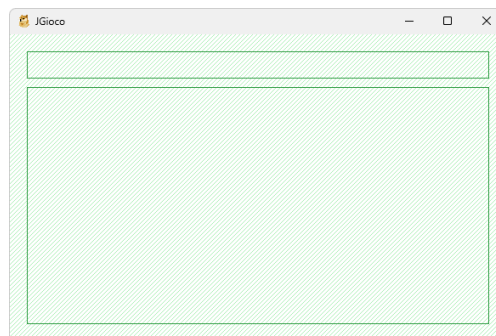
Uno dei modi per rappresentare i colori è il modello RGBA, che include il **canale alpha**. Un altro modo è tramite la rappresentazione esadecimale ❹.

Quando aggiungo un elemento ad un container, posso specificare come deve essere trattato tramite il metodo `add(Component comp, Object constraints)`: in base al layout del container, `Object constraints` avrà un significato diverso.

ⓘ Nota

`BorderFactory` ❷ è un esempio di Factory Pattern

Risultato



Implementazione

```
public class App extends JFrame {
    static {
        UIManager.put(
            "Label.font",
            new Font("Cascadia Code", Font.PLAIN, 17)
        );
        UIManager.put("Label.foreground", Color.decode("#2f9e44"));
        UIManager.put("Label.background", Color.WHITE);

        UIManager.put(
            "Button.font",
            new Font("Cascadia Code", Font.PLAIN, 17)
        );
    }
}
```

```

);
UIManager.put("Button.foreground", Color.decode("#2f9e44"));
UIManager.put("Button.background", Color.WHITE);

UIManager.put("Button.border",
BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(Color.decode("#2f9e44")),
    BorderFactory.createEmptyBorder(5, 10, 5, 10)));

UIManager.put("Button.highlight", Color.decode("#b2f2bb"));
UIManager.put("Button.select", Color.decode("#b2f2bb"));
UIManager.put("Button.focus", Color.WHITE);

UIManager.put("Panel.background", Color.WHITE);
}

```

```

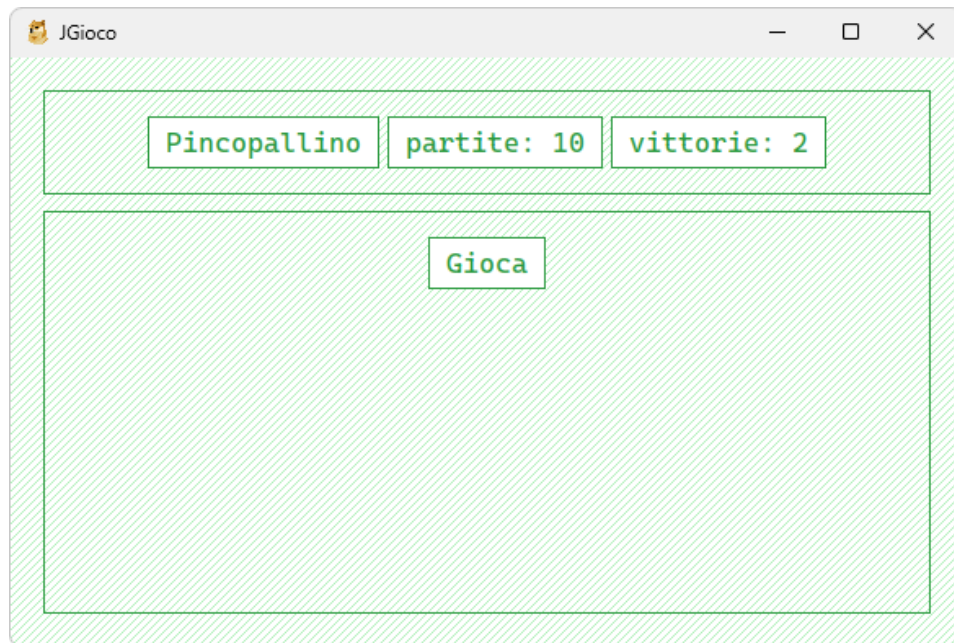
App() {
    add(new JPanel(new BorderLayout(10, 10)) {
        {
            // ...
            add(new JPanel() {
                {
                    // ...
                    String[] labels = { "Pincopallino", "partite: 10",
"vittorie: 2" };
                    for (String label : labels)
                        add(new JLabel(label) {
                            {
                                setOpaque(true);
                                setBorder(BorderFactory.createCompoundBorder(
BorderFactory.createLineBorder(Color.decode("#2f9e44")),
                                BorderFactory.createEmptyBorder(5, 10, 5, 10)));
                            }
                        });
                }
            }, BorderLayout.NORTH);

            add(new JPanel() { { ...; add(new JButton("Gioca")); } },
BorderLayout.CENTER);
        }

        @Override
        protected void paintComponent(Graphics g) { ... }
    });
    // ...
}

public static void main(String[] args) { new App(); }
}

```



Implementazione pt.2

```
public class App extends JFrame {
    static { /* Usate il blocco alla pagina 12 */ }

    Border simpleBorder(int horizontal, int vertical) {
        return BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(Color.decode("#2f9e44")),
            BorderFactory.createEmptyBorder(horizontal, vertical,
horizontal, vertical));
    }

    App() {
        super("JGioco");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(600, 400);
        try { setIconImage(ImageIO.read(new File("icon.png"))); }
        catch (IOException e) { }

        add(new JPanel(new BorderLayout(10, 10)) {
            {
                setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
                add(new JPanel() {
                    {
                        setBackground(new Color(255, 255, 255, 0));

                        setBorder(simpleBorder(10, 10));
                        Stream.of(new String[] { "Pincopallino", "partite:
10", "vittorie: 2" })
                            .map(label -> new JLabel(label) {
```



```

        {
            setOpaque(true);
            setBorder(simpleBorder(5, 10));
        }
    }).forEach(this::add);
}
}, BorderLayout.NORTH);

add(new JPanel() {
    {
        setBackground(new Color(255, 255, 255, 0));
        setBorder(simpleBorder(10, 10));
        add(new JButton("Gioca"));
    }
}, BorderLayout.CENTER);
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    int density = 5;
    g.setColor(Color.decode("#b2f2bb"));
    for (int x = 0; x ≤ getWidth() + getHeight(); x += density)
        g.drawLine(x, 0, 0, x);
}
});

setLocationRelativeTo(null); setVisible(true);
}

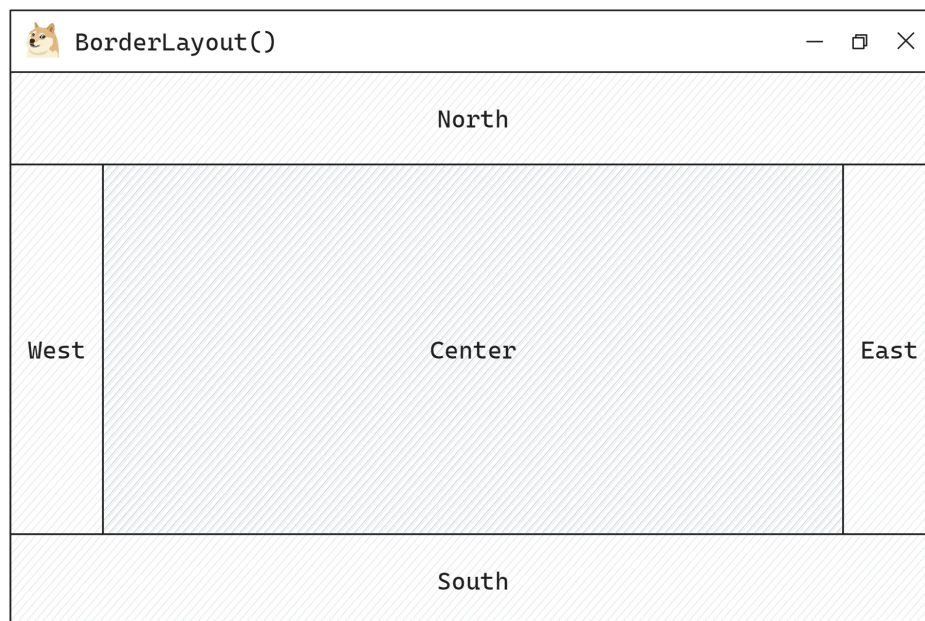
public static void main(String[] args) { new App(); }
}

```

Funzionamento del BorderLayout

Il BorderLayout permette di specificare la regione del Container occupata da ciascun componente:

- il componente posizionato in CENTER occupa tutto lo spazio disponibile.
- i componenti a NORTH e SOUTH occupano la larghezza massima (indipendentemente dalla larghezza impostata) e occupano l'altezza minima, o, se impostata, l'altezza impostata.
- i componenti WEST e EAST occupano l'altezza massima (indipendentemente dall'altezza impostata) e occupano la larghezza minima, o, se impostata, la larghezza impostata.



```
class DecoratedPanel { ... } // Per esercizio definirla :)

public class App extends JFrame {
    App() {
        add(new JPanel(new BorderLayout()) { {
            add(new DecoratedPanel("Nord"), BorderLayout.NORTH);
            add(new DecoratedPanel("Sud"), BorderLayout.SOUTH);
            add(new DecoratedPanel("West") {
                { setPreferredSize(new Dimension(120, 0)); } // Larghezza
custom
                }, BorderLayout.WEST);
            add(new DecoratedPanel("East"), BorderLayout.EAST);
            add(new DecoratedPanel("Center"), BorderLayout.CENTER);
        } });
    }

    public static void main(String[] args) { new App(); }
}
```

[CardLayout](#) ([Java API doc](#))

Menu, impostazioni e partita (cambiare schermata)

Il [CardLayout](#) è molto utile quando abbiamo più schermate (menu principale, impostazioni, partita etc...) e si vuole un modo **semplice** per passare da una schermata all'altra.

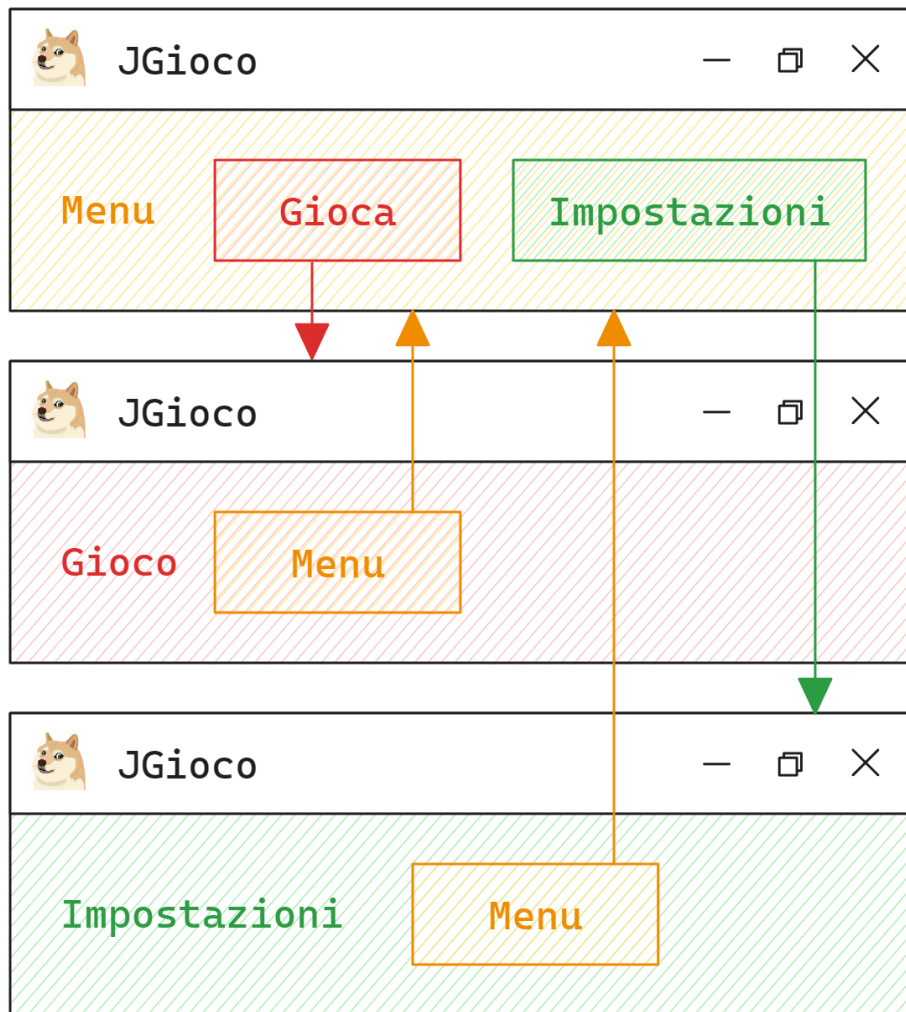


Figura 3: caso d'uso di un [CardLayout](#) e **frecce** nel **wireframe** per le transizioni

L'implementazione più rozza

```
public class App extends JFrame {
    App() {
        super("JGioco");
        // ...

        JPanel panel;

        add(panel = new JPanel(new CardLayout()) {
            {
                add(new MenuPanel() ❶, "Menu" x2);
                add(new SettingsPanel() ❶, "Settings" x2);
                add(new GamePanel() ❶, "Game" x2);
            }
        });

        ❸ ((CardLayout) panel.getLayout()).show(panel, "Menu");
        // ...
    }

    public static void main(String[] args) { new App(); }
}
```

Ad ogni schermata ❶ bisogna associare un **nome** ❷ quando viene aggiunta al `JPanel` con il `CardLayout`. Per visualizzare la schermata scelta basta usare il metodo `show(Container parent, String name)` del `CardLayout` ❸

Questo approccio ha 2 problemi:

- è facile **sbagliare il nome** della schermata, essendo una stringa
- non c'è un elenco esplicito delle schermate disponibili

Usando gli enum

Per ovviare a questi problemi, si può usare un `enum`

```
enum Screen { Menu, Settings, Game }

public class App extends JFrame {
    App() {
        super("JGioco");
        JPanel panel;
        add(panel = new JPanel(new CardLayout()) {
            {
                add(new MenuPanel() ❶, Screen.Menu.name());
                add(new SettingsPanel() ❶, Screen.Settings.name());
                add(new GamePanel() ❶, Screen.Game.name());
            }
        });

        ((CardLayout) panel.getLayout()).show(panel,
        Screen.Game.name());
    }
    public static void main(String[] args) { new App(); ❷ }
}
```

Il problema è che, per poter **cambiare schermata**, bisogna passare ai vari componenti ❶ l'istanza di **App** ❷ di cui si vuole cambiare la schermata, creando un groviglio di **spaghetti code**. Ma c'è una soluzione per ovviare anche a questo problema.

Singleton + Observer

```
enum Screen { Menu, Settings, Game }

@SuppressWarnings("deprecation")
❶ class Navigator extends Observable {
    private static Navigator instance;

    private Navigator() { }

    public static Navigator getInstance() {
        if (instance == null)
            instance = new Navigator();
        return instance;
    }

    public void navigate(Screen screen) {
        setChanged();
        notifyObservers(screen);
    }
}

@SuppressWarnings("deprecation")
❷ public class App extends JFrame implements Observer {
    JPanel panel;

    App() {
        /* ... */
        ❸ Navigator.getInstance().addObserver(this);

        add(panel = new JPanel(new CardLayout())) {
            {
                add(new MenuPanel(), Screen.Menu.name());
                add(new SettingsPanel(), Screen.Settings.name());
                add(new GamePanel(), Screen.Game.name());
            }
        };
        /* ... */
    }

    @Override
    ❹ public void update(Observable o, Object arg) {
        if (o instanceof Navigator && arg instanceof Screen)
            ❺ ((CardLayout) panel.getLayout()).show(panel, ((Screen)
arg).name());
    }

    public static void main(String[] args) {
```

```

    new App();
    6 Navigator.getInstance().navigate(Screen.Settings);
  }
}

```

- **Navigator** ❶ è la classe che permette di cambiare schermata
 - usa il pattern **Singleton** perché deve avere una sola istanza globale
 - usa il pattern **Observer** per segnalare alla vista il cambiamento di schermata
 - per cambiare schermata, basta usare ❷
- **App** ❷
 - è un **Observer** perché deve ricevere le segnalazioni da **Navigator** ❶
 - usa ❸ per osservare l'unica istanza di **Navigator** ❶
 - nel metodo ❹ cambia la schermata usando **CardLayout::show** ❺

🕒 Nota

L'utilizzo del **Singleton Pattern** in questo caso è un po' esagerato. Si potrebbe benissimo creare un'istanza di **Navigator** e passarla ai vari componenti, come nel progetto Minesweeper (permettendo di avere più **Navigator** diversi).

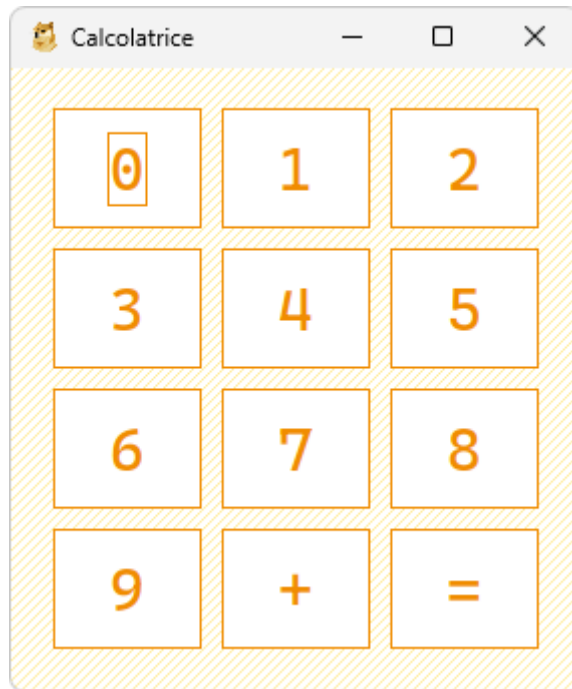
Il lettore è incoraggiato a valutare l'opzione che ritenete più adatta al suo progetto, o di crearne un'altra :)

[GridLayout](#) (Java API doc)

Non è un layout particolarmente complesso: permette di specificare il numero di righe, il numero di colonne, e lo spazio fra le righe e le colonne.

```
frame.add(new JPanel(new GridLayout(4, 3, 10, 10)) {
    {
        setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));

        for (int digit = 0; digit ≤ 9; digit++)
            add(new JButton(String.valueOf(digit)));
        add(new JButton("+"));
        add(new JButton("="));
    }
});
```



Implementazione

```
public class App extends JFrame {
    static {
        Color YELLOW = Color.decode("#f08c00");

        UIManager.put("Button.font", new Font("Casadia Code",
        Font.PLAIN, 30));
        UIManager.put("Button.foreground", YELLOW);
        UIManager.put("Button.background", Color.WHITE);
    }
}
```

```

        UIManager.put("Button.border",
            BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(YELLOW),
                BorderFactory.createEmptyBorder(10, 15, 10, 15)));
        UIManager.put("Button.highlight", Color.decode("#ffec99"));
        UIManager.put("Button.select", Color.decode("#ffec99"));
        UIManager.put("Button.focus", YELLOW);
        UIManager.put("Panel.background", Color.WHITE);
    }

    App() {
        super("Calcolatrice");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        try {
            setIconImage(ImageIO.read(new File("icon.png")));
        } catch (IOException e) {
        }

        add(new JPanel(new GridLayout(4, 3, 10, 10)) {
            {
                setBorder(
                    BorderFactory.createEmptyBorder(20, 20, 20, 20));
                for (int digit = 0; digit ≤ 9; digit++)
                    add(new JButton(String.valueOf(digit)));
                add(new JButton("+"));
                add(new JButton("="));
            }

            @Override
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);
                int d = 5;
                g.setColor(Color.decode("#ffec99"));
                for (int x = 0; x ≤ getWidth() + getHeight(); x += d)
                    g.drawLine(x, 0, 0, x);
            }
        });

        setSize(300, 350);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    public static void main(String[] args) { new App(); }
}

```


[GridBagLayout](#) ([Java API doc](#))

Il layout **più flessibile**

Il [GridBagLayout](#) divide il panel in una griglia dinamica e permette di controllare la **posizione** dei componenti **all'interno di ogni singola cella**:

- una cella può essere **posizionata** in qualunque x e y della griglia
- la **dimensione** di una cella può essere definita in due modi:
 - specificando numero di **righe** e numero di **colonne** da occupare
 - specificando **regole dinamiche** per ridimensionare larghezza e altezza

La particolarità (rispetto agli altri layout) è che i componenti all'interno delle celle **non vengono ridimensionati** e si possono **posizionare liberamente** (non occupano tutto lo spazio della cella).

Ad esempio, nella cella a sinistra si può vedere che il label "testo" è posizionato a **NORTH_EAST**.

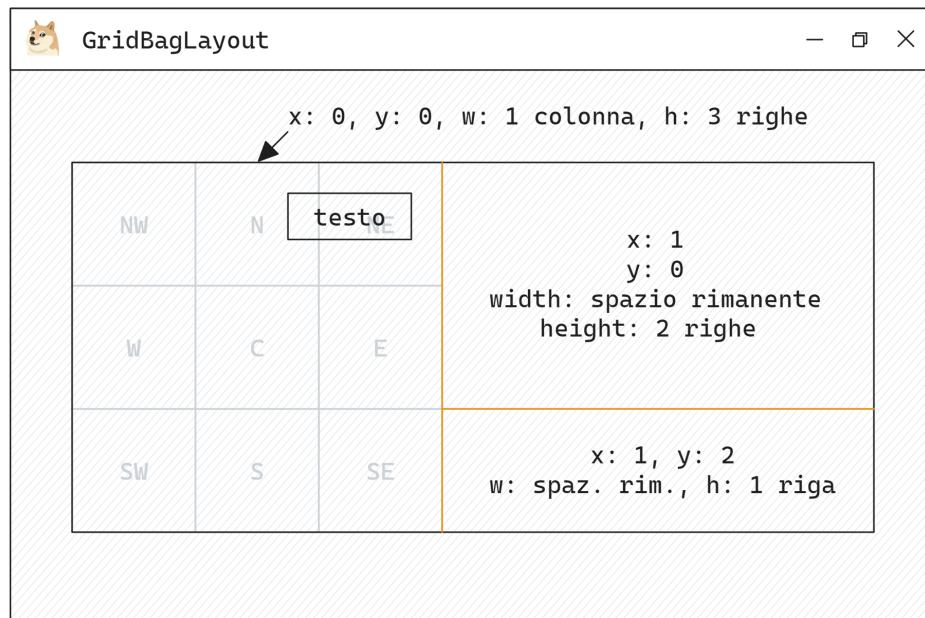


Figura 4: caso d'uso di un [GridBagLayout](#)

Implementazione

```
public class App extends JFrame {
    static Color TRANSPARENT = new Color(0, 0, 0, 0);

    static {
        UIManager.put("Label.font", new Font("Casadia Code",
Font.PLAIN, 14));
        UIManager.put("Panel.background", Color.WHITE);
    }

    App() {
        super("JGioco");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        try {
            setIconImage(ImageIO.read(new File("icon.png")));
        } catch (IOException e) {
        }

        JFrame frame = this;
        setSize(600, 400);

        add(new JPanel(new GridLayout(1, 1)) {
            {
                setBorder(BorderFactory.createEmptyBorder(50, 30, 50, 30));

                add(new JPanel(new GridBagLayout()) {
                    {
                        setBackground(TRANSPARENT);
                        setBorder(
                            BorderFactory.createCompoundBorder(
                                BorderFactory.createLineBorder(Color.DARK_GRAY),
                                BorderFactory.createEmptyBorder(30, 30, 30, 30)));

                        GridBagConstraints c = new GridBagConstraints();

                        c.weightx = 1;
                        c.weighty = 1;

                        c.anchor = GridBagConstraints.NORTHEAST;
                        c.gridx = 0;
                        c.gridy = 0;
                        c.gridwidth = 1;
                        c.gridheight = 3;

                        add(new JLabel("testo"), c);

                        c.anchor = GridBagConstraints.CENTER;
                        c.gridx = 1;
                        c.gridy = 0;
                        c.gridwidth = GridBagConstraints.REMAINDER;
                        c.gridheight = 2;
```

```

        add(new JLabel(
            "x: 1,\n y: 0, w: spaz. rim., h: 2 righe"), c);

        c.gridx = 1;
        c.gridy = 2;
        c.gridwidth = GridBagConstraints.REMAINDER;
        c.gridheight = 1;

        add(new JLabel(
            "x: 1, y: 2, w: spaz. rim., h: 1 riga"), c);
    }

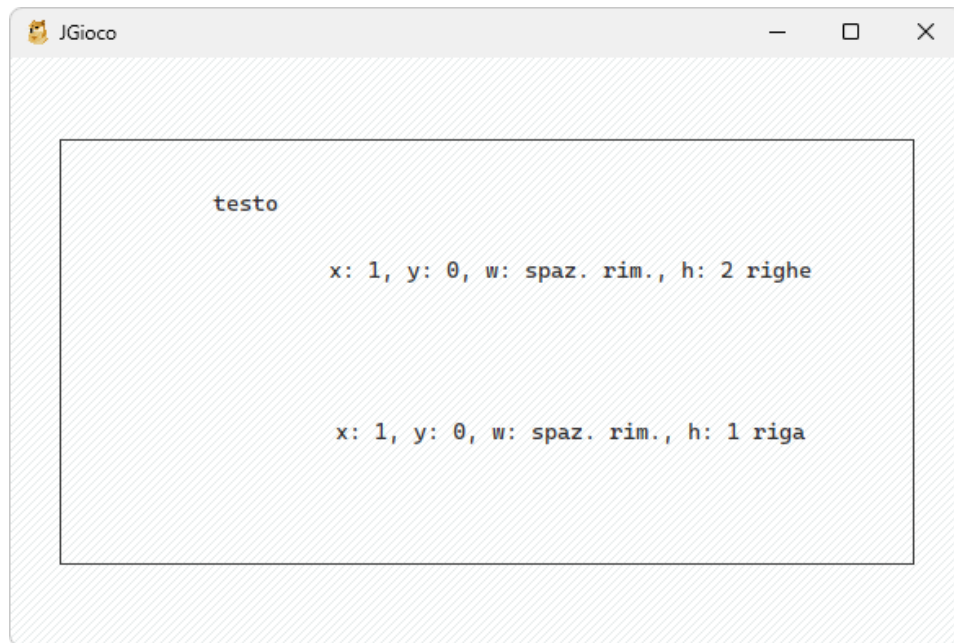
    @Override
    protected void paintComponent(Graphics g) {
        setPreferredSize(
            new Dimension(frame.getWidth() - 80,
                frame.getHeight() - 80));
        super.paintComponent(g);
    }
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    int d = 5;
    g.setColor(Color.decode("#e9ecef"));
    for (int x = 0; x ≤ getWidth() + getHeight(); x += d)
        g.drawLine(x, 0, 0, x);
}
});

setLocationRelativeTo(null);
setVisible(true);
}

public static void main(String[] args) { new App(); }
}

```



`GridBagConstraints` ha anche **altri attributi** estremamente utili come `ipdax`, `ipady` e `insets` che sono discussi nella [guida ufficiale](#)

In conclusione (sui Layout)

Con questi strumenti è possibile coprire la maggior parte delle necessità che riguardano l'implementazione di un **wireframe**, in modo semplice e pulito.

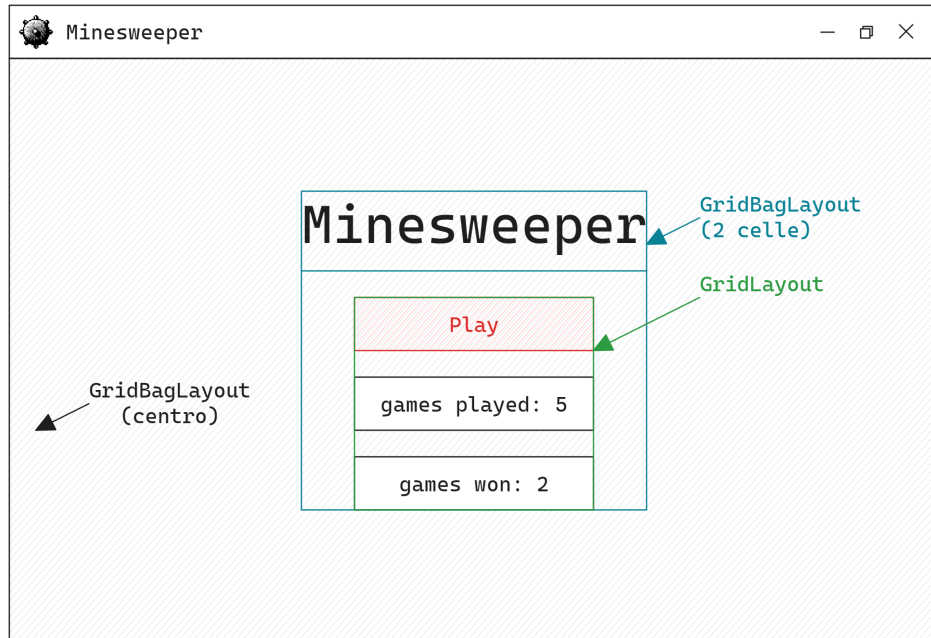


Figura 5: esempio di wireframe per il gioco «Minesweeper»

I layout visti in questa guida non sono gli unici (e non ne sono state coperte tutte le funzionalità), ma, capiti i concetti chiave, si può consultare la [documentazione](#).

Pulsanti e Functional Interfaces

Non c'è molto da dire sui pulsanti: sono dei componenti che invocano un metodo quando vengono premuti. Una delle funzionalità che molti non usano nel contesto dei pulsanti è quella delle **SAM** (*Single Abstract Method*) **Functional Interfaces**.

```
JButton button = new JButton("Pulsante");
button.addActionListener(① new ActionListener() {
    @Override
    ② public void actionPerformed(ActionEvent e) {
        System.out.println("Pulsante premuto!");
    }
});

panel.add(button);
```

Per eseguire del codice quando viene premuto il pulsante bisogna aggiungerli un `ActionListener` **①**, in cui si sovrascrive il metodo `actionPerformed` **②**.

```
package java.awt.event;

import java.util.EventListener;

public interface ActionListener extends EventListener {
    ① void actionPerformed(ActionEvent var1);
}
```

Analizzando l'implementazione di `ActionListener` notiamo che si tratta di un'interfaccia con **un solo metodo astratto** **①**, quindi è possibile semplificare il codice sopra in questo modo:

```
JButton button = new JButton("Pulsante");
button.addActionListener(e → System.out.println("Pulsante
premutato!"));

panel.add(button);
```

Se si vogliono usare gli **instance initialization block**, è possibile scrivere direttamente:

```
panel.add(new JButton("Pulsante") {{
    addActionListener(e → System.out.println("Pulsante premuto!"));
}});
```

Disegnare programmaticamente con le trasformazioni

Semplificare il codice con `.translate(int x, int y)`

Si supponga di voler disegnare dei quadrati posizionandoli in diagonale partendo da (0, 0), il codice non è particolarmente complesso:

```
class Panel extends JPanel {
    Panel() {
        super();
        setPreferredSize(new Dimension(200, 300));
    }

    @Override
    public void paint(Graphics g) {
        super.paint(g);

        g.setColor(Color.CYAN);
        g.fillRect(0, 0, 200, 300);

        // Disegna una diagonale in posizione (0, 0)
        for (int position = 0; position < 10; position++) {
            g.setColor(Color.YELLOW);
            g.fillRect(position * 10, position * 10, 10, 10);

            g.setColor(Color.RED);
            g.drawRect(position * 10, position * 10, 10, 10);
        }
    }
}
```

Si supponga di voler disegnare questa diagonale in una posizione arbitraria, ad esempio (60, 110), è possibile modificare il codice aggiungendo gli offset: ❶

```
class Panel extends JPanel {
    Panel() { super(); setPreferredSize(new Dimension(200, 300)); }

    @Override
    public void paint(Graphics g) {
        super.paint(g);

        g.setColor(Color.CYAN);
        g.fillRect(0, 0, 200, 300);

        ❶ int offsetX = 60, offsetY = 110;

        // Disegna una diagonale in posizione (60, 110)
        for (int position = 0; position < 10; position++) {
            g.setColor(Color.YELLOW);
            g.fillRect(offsetX ❶ + position * 10, offsetY x1 + position
                * 10, 10, 10);

            g.setColor(Color.RED);
        }
    }
}
```

```

        g.drawRect(offsetX ❶ + position * 10, offsetY x1 + position
* 10, 10, 10);
    }
}
}

```

0, peggio ancora, si potrebbero inserire gli offset a mano. Ma c'è uno strumento molto più comodo per semplificare il lavoro: `Graphics::translate` ❶.

```

class Panel extends JPanel {
    Panel() {
        super();
        setPreferredSize(new Dimension(200, 300));
    }

    @Override
    public void paint(Graphics g) {
        super.paint(g);

        g.setColor(Color.CYAN);
        g.fillRect(0, 0, 200, 300);

        g.translate(60, 110); ❶

        // Disegna una diagonale in posizione (0, 0)
        for (int position = 0; position < 10; position++) {
            g.setColor(Color.YELLOW);
            g.fillRect(position * 10, position * 10, 10, 10);

            g.setColor(Color.RED);
            g.drawRect(position * 10, position * 10, 10, 10);
        }

        g.translate(-60, -110); ❷
    }
}

```

Con `translate` ❶ è possibile spostare l'origine da cui disegnare. Quindi si può usare il codice relativo a (0, 0) e traslarlo alla posizione (60, 110). Questa tecnica è molto comoda quando bisogna disegnare oggetti più complessi.

Dopo aver disegnato la diagonale si usa `g.translate(-60, -110)` ❷ per riportare l'origine a (0, 0) per il resto del codice.

Transformazioni affini

Castando `Graphics` g a `Graphics2D` si hanno anche più opzioni a disposizione per controllare la grafica:

- traslazione
- scala ❷
- rotazione ❸
- taglio

Con `AffineTransform` c'è anche un modo più comodo per resettare la grafica ❶.

```
class Panel extends JPanel {
    Panel() {
        super();
        setPreferredSize(new Dimension(200, 300));
    }

    @Override
    public void paint(Graphics g) {
        super.paint(g);

        Graphics2D g2d = (Graphics2D) g;
        AffineTransform defaultTransform = g2d.getTransform() ❶;

        g2d.setColor(Color.ORANGE);
        g2d.fillRect(0, 0, 200, 300);

        g2d.scale(2, 2) ❷;
        g2d.rotate(0.1) ❸;

        for (int position = 0; position < 10; position++) {
            g2d.setColor(Color.YELLOW);
            g2d.fillRect(position * 10, position * 10, 10, 10);

            g2d.setColor(Color.RED);
            g2d.drawRect(position * 10, position * 10, 10, 10);
        }

        g2d.setTransform(defaultTransform) ❶;

        for (int position = 0; position < 10; position++) {
            g2d.setColor(Color.YELLOW);
            g2d.fillRect(position * 10, position * 10, 10, 10);

            g2d.setColor(Color.RED);
            g2d.drawRect(position * 10, position * 10, 10, 10);
        }
    }
}
```

Con questi strumenti si può scalare molto facilmente la parte grafica del progetto in base alla dimensione del `JFrame` o in base alle impostazioni dell'utente. Inoltre, `scale()` e `rotate()` sono molto comodi per fare semplici animazioni.

Timer

◉ Nota

Questa sezione la devo ultimare 😊. Probabilmente, quando sarà completa, sarà molto semplice, con giusto un paio d'esempi e una piccola spiegazione teorica sulle differenze fra i vari `Timer`

Nello sviluppo di un gioco ci sono alcune cose che vorremmo fare:

- ritardare un'azione
- animazioni
- impostare un timer globale (`frameRate`)
 - per sincronizzare le animazioni
 - per fare i calcoli
 - etc...

Diversi tipi di `Timer` paragonati

- `java.util.Timer`
- `java.swing.Timer`
- `java.util.concurrent.ScheduledExecutorService`

`java.util.Timer` solo un thread `java.util.conc...` etc... schedula le varie cose in modo da evitare quello che si chiama **head of line block**

◉ Nota

Un esempio di `ScheduledExecutorService` [qui](#)

Ritardare azioni

Animazioni

Framerate del gioco

In conclusione su Java Swing

Sono state discusse solo **alcune** delle funzionalità di Swing (quelle che ho notato servano più spesso nei progetti). In questa guida non è stato approfondito (in particolare dal punto di vista teorico) come si comporta Swing e cos'è in grado di fare.

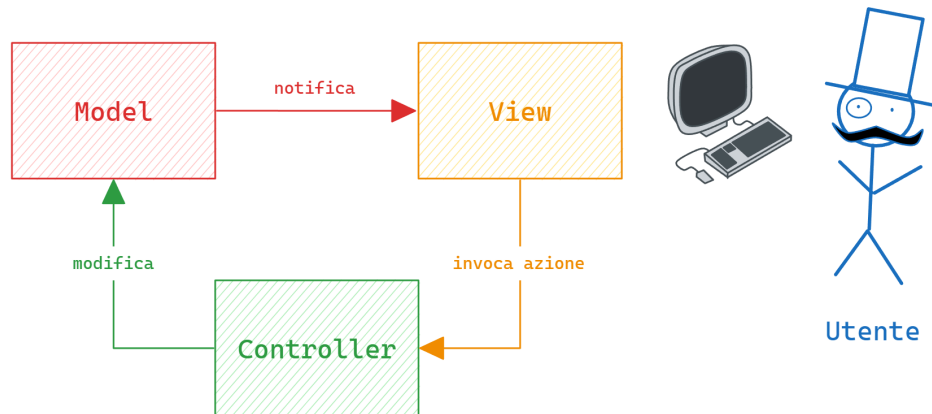
Suggerisco di consultare l'API docs di Java <https://docs.oracle.com/en/java/javase/21/docs/api/index.html> per altre informazioni.

Tutto quello che è stato discusso in questa prima parte del progetto riguarda **solamente** la **View**, ci sono altre due componenti importanti da vedere nel pattern architetturale **MVC** che verranno discusse nella sezione successiva.

◉ Nota

Si invita il lettore a reinterpretare il codice presentato e adattarlo al proprio progetto e **stile** :)

MVC



MVC è un **pattern architetturale** per sviluppare diversi tipi di applicazioni: web app, interfacce grafiche, TUI (Terminal User Interface) etc... L'idea è quella di **separare** la logica del programma dall'interfaccia. In questo modo la logica è **definita in modo chiaro e pulito**, ed è **riusabile** (più interfacce diverse possono condividere la stessa logica)

Ad esempio, è possibile definire la logica del gioco Solitario una volta sola, e usarla per costruire un sito web, un'applicazione per Windows, un'API REST etc... Il **Model** è condiviso da più **View**.

Model

Il **Model** contiene **esclusivamente la logica** del programma: le **entità**, come queste **interagiscono** fra di loro e quali sono gli **stati validi** del programma.

View

Nella sezione su **Java Swing** si è visto solo come progettare e implementare una **View**, ora l'obiettivo è quello di progettare e implementare un'applicazione completa, mostrando un possibile modo di strutturare il codice per far interagire il Model con l'interfaccia grafica.

Controller

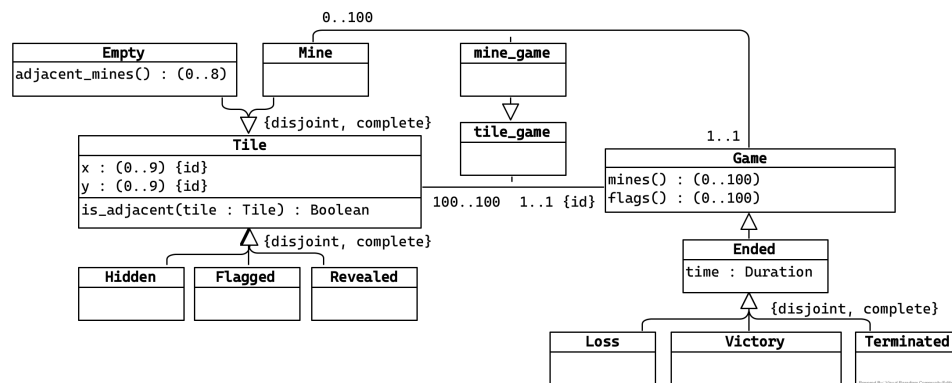
Il **Controller** è il collante fra l'interfaccia e il Model. Ci sono diverse strategie per implementarlo, in questa guida ne sarà discussa una particolarmente semplice.

Minesweeper (prato fiorito)

Per mostrare come progettare Model e Controller verrà discussa l'analisi e l'implementazione di un semplice progetto: **Minesweeper** (il cui codice si trova [su GitHub](#))

UML e modello

L'analisi inizia con la raccolta dei **requisiti** e la **realizzazione di un modello** del problema (in questo esempio i requisiti sono le regole e le funzionalità del gioco **Minesweeper**). Di seguito un possibile modello per il gioco:



Nota

ATTENZIONE! Questo **NON** è il tipo di UML che avete visto a lezione. Questo è un modello più formale (legato alla logica di primo ordine). Si vede in corsi come **Basi di Dati 2**. È più semplice ed astrae i dettagli implementativi del progetto, ma comunque abbastanza potente da permettere di ragionare sulle funzionalità che si vogliono implementare.

Questo modello rappresenta i seguenti requisiti:

- vogliamo gestire le partite (Game)
 - ogni partita ha 100 caselle (Tile)
 - sono disposte in una griglia 10x10
 - c'è un numero variabile di mine da 0 a 100
 - una partita può essere finita (Ended)
 - caso in cui va segnata la durata
 - può avere uno di tre esiti
 - sconfitta (Loss)
 - vittoria (Victory)
 - terminata dall'utente (Terminated)
 - di ogni partita si devono poter calcolare il numero di mine e il numero di bandiere
- ogni casella (Tile)
 - può essere di uno dei due tipi
 - vuota (Empty)
 - con una mina (Mine)
 - può trovarsi in uno dei tre stati
 - nascosta (Hidden)
 - con una bandiera (Flagged)

- scoperta (Revealed)
- di ogni casella si devono poter calcolare le caselle adiacenti
- di ogni casella vuota si deve conoscere il numero di mine vicine

Vincoli del modello

Il modello definito sopra è **incompleto**: permette **alcuni stati invalidi**:

- una partita potrebbe essere vinta anche con una mina scoperta
- una partita potrebbe essere vinta con tutte le caselle nascoste

Servono dei vincoli (qui scritti in *logica di primo ordine*) per **limitare** i possibili stati del modello. Di seguito due esempi di vincolo.

◉ Nota

Di seguito sono riportati alcuni vincoli, [qui è possibile consultare il documento completo](#)

[Constraint.**Game**.victory_condition]

description

Una partita è vinta in uno di due casi:

- tutte e sole le caselle con una mina hanno una bandiera
- tutte le caselle vuote sono scoperte

constraint

$$\begin{aligned} \forall \text{ game } & \mathbf{Victory}(\text{game}) \iff \\ & \forall \text{ tile } \text{mine_game}(\text{tile}, \text{game}) \implies \mathbf{Flagged}(\text{tile}) \wedge \\ & \neg \exists \text{ tile } \\ & \quad \text{tile_game}(\text{tile}, \text{game}) \wedge \\ & \quad \mathbf{Empty}(\text{tile}) \wedge \\ & \quad \mathbf{Flagged}(\text{tile}) \\ \vee \\ & \forall \text{ tile } \\ & \quad (\text{tile_game}(\text{tile}, \text{game}) \wedge \mathbf{Empty}(\text{tile}) \\ & \quad \implies \mathbf{Revealed}(\text{tile})) \end{aligned}$$

[Constraint.**Game**.loss_condition]

description

Una partita è una sconfitta se e solo se c'è una mina scoperta

constraint

$$\begin{aligned} \forall \text{ game } & \mathbf{Loss}(\text{game}) \\ & \iff \\ & \exists \text{ mine } \text{mine_game}(\text{mine}, \text{game}) \wedge \mathbf{Revealed}(\text{mine}) \end{aligned}$$

◉ Nota

In Java questi vincoli sono spesso implementabili con gli [Stream](#)

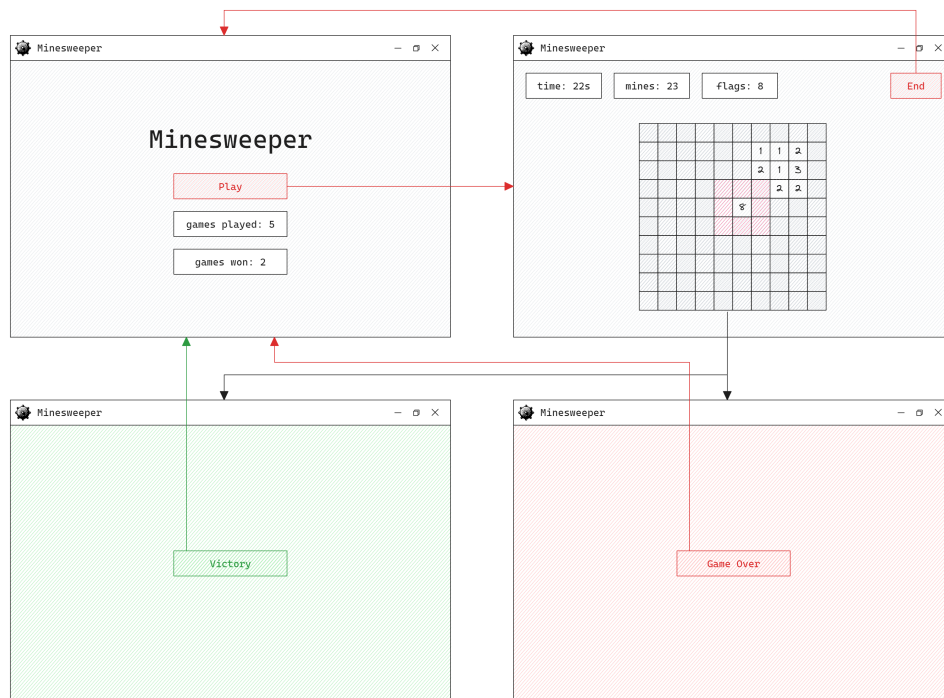
Operazioni sul modello

Bisogna definire un insieme di **operazioni** che un **utente** (o un altro sistema!) deve poter effettuare sul modello. Queste operazioni sono l'interfaccia (l'API) che il Model dovrà fornire a chi lo vuole usare.

- `start_game()`: **Game**
- `terminate_game(game: Game)`: **Terminated**
- `reveal(tile: Hidden)`: **Revealed**
- `flag(tile: Hidden)`: **Flagged**
- `remove_flag(tile: Flagged)`: **Hidden**
- `games_played()`: **Integer** ≥ 0
- `games_won()`: **Integer** ≥ 0
- `time()`: **Duration**
- `mines()`: (0..100)
- `flags()`: (0..100)

Wireframe

A questo punto rimane solo da progettare l'interfaccia dell'applicazione con un **wireframe** (garantendo le operazioni definite sopra)



Codice

Nel codice del progetto ho deciso di creare un **package** principale chiamato **minesweeper** e di suddividerlo a sua volta in 3 **package** per **model**, **view** e **controller**. Il progetto è piccolo, quindi bastano questi tre. I **package** sono lo strumento più importante per l'**encapsulation**.

Model

Di seguito una breve analisi delle classi del model.

```
package minesweeper.model;

import static minesweeper.model.Tile.Visibility.*;
import java.util.Observable;
import java.util.Optional;

@SuppressWarnings("deprecation")
public class Tile extends Observable ❶ {
    ❷ public enum Visibility { Hidden, Flagged, Revealed }

    public final int x, y ❸;
    public final boolean isMine;
    private Visibility visibility = Hidden;
    ❹ Optional<Integer> adjacentMines = Optional.empty();

    ❺ Tile(int x, int y, boolean isMine) {
        this.x = x;
        this.y = y;
        this.isMine = isMine;
    }

    public Visibility visibility() { return visibility; }

    ❻ public Optional<Integer> adjacentMines() { return
adjacentMines; }

    public void reveal() {
        if (visibility != Hidden)
            return;

        ❽ setChanged();
        notifyObservers(❻ visibility = Revealed);
    }

    ❼ public void flag() {
        ❾ setChanged();
        notifyObservers(❻ visibility = x8 switch (visibility) {
            case Hidden → Flagged;
            case Flagged → Hidden;
            case Revealed → {
                clearChanged();
                yield Revealed;
            }
        })
    }
}
```



```

    });
  }
}

```

Encapsulation con `public final`

Un modo alternativo per ottenere **encapsulation** è quello di segnare un **attributo** come `public final` ③, e di lasciare al **costruttore** la **visibilità di default** ⑤. In questo modo:

- `model.Tile` può essere istanziata solo con valori validi all'interno del package
- questi valori possono essere letti ma non modificati

Tipizzazione con `enum`

I tipi di visibilità delle `Tile` (`Hidden`, `Flagged` e `Revealed`) sono definiti tramite `enum` ②. Il tipo `Visibility` è definito **internamente** alla classe `Tile` per poter usare la notazione `Tile.Visibility` che trovo più leggibile. Inoltre non serve creare un altro file per questo `enum`, facilitando la **navigabilità del codice**.

Valori indefiniti: `null` vs `Optional<T>`

In alcuni casi **ha senso ed è utile** avere attributi che potrebbero essere **indefiniti** (`adjacentMines` ④ non è definito se la `Tile` è una mina).

Usare `null` è scomodo: chi usa la libreria **deve scoprire**, tramite la documentazione, che quell'attributo potrebbe non essere definito.

Con `Optional<T>` si **dichiara esplicitamente nel codice** che l'attributo potrebbe essere indefinito (`Optional.empty()`), e chi usa l'attributo **deve gestire** il caso in cui il valore non c'è.

Le `switch expression` (`switch con steroidi`)

In Java 14 sono state ufficializzate le `switch expression` ⑧ che possono restituire un valore e non necessitano dell'utilizzo di `break`; (vedere il metodo `flag()` ⑦).

Gli assegnamenti come espressioni

In Java gli assegnamenti alle variabili sono espressioni: ad esempio, `y = (x = 5)` assegna 5 a x, e ritorna 5 per assegnarlo a y ⑥.

Observer Pattern

Per la casella ho deciso di usare l'**Observer Pattern** ① per due motivi:

- notificare la View (per ridisegnare la casella)
- notificare le altre classi del Model (la classe `Game` deve sapere quando una `Tile` cambia stato, per decidere se terminare la partita o rivelare le caselle adiacenti)

ⓘ Nota

Per notificare gli Observer, bisogna usare `setChanged` ⑨ prima di inviare la notifica.

```

@SuppressWarnings("deprecation")
public class Game extends Observable implements Observer {

    // ...

    final Tile[] tiles = new Tile[100];
    public final int mines;
    int flags = 0;
    Duration time = Duration.ofSeconds(0);

    public Game() {
        Random random = new Random(); ❶

        for (int y = 0; y < 10; y++)
            for (int x = 0; x < 10; x++)
                tiles[y * 10 + x] = new Tile(x, y, random.nextInt(100) ≥
15 ? Empty : Mine);

        for (Tile tile : tiles) {
            tile.addObserver(this);

            int adjacentMines = (int) ❷ adjacent(tile.x, tile.y)
                .filter(t → t.kind == Mine)
                .count();

            if (tile.kind == Empty && adjacentMines > 0)
                tile.adjacentMines = Optional.of(adjacentMines);
        }

        mines = (int) ❷ Stream.of(tiles)
            .filter(t → t.kind == Mine)
            .count();
    }

    public void updateTime() {
        time = time.plusSeconds(1);
        setChanged();
        notifyObservers(Message.Timer);
    }

    // ...

```

Un errore che ho visto in vari progetti è quello di creare un attributo per ogni variabile. Ad esempio: alcuni hanno creato un attributo `Random random` ❶ per la classe `Game`, nonostante questo venga usato solo nel costruttore ❶. **NON** c'è bisogno di **creare un attributo per ogni variabile**.

Alcuni esempi di `Stream`

Nella classe `model.Game` ci sono alcuni esempi di `Stream` ❷.

Gestione del timer

Il **Model** non dovrebbe gestire l'eventuale **timer**, quello è un compito del **Controller**. Mettendo il **timer** fuori dal **Model** diventa più facile mettere in **pausa** il gioco.

Il **Model** dovrebbe semplicemente avere un metodo `update()` (nel mio caso `updateTime()`) che deve essere invocato dal **Controller** quando scatta il **timer**.

Il metodo `update(Observable o, Object arg)` è instanceof

```
@Override
public void update(Observable o, Object arg) {
    if (!(o instanceof Tile tile ❶ && arg instanceof Visibility
visibility x1))
        return;

    setChanged();

    switch (visibility) {
        case Flagged → flags++;
        case Hidden → flags--;
        case Revealed → {
            if (tile.isMine ❷) {
                notifyObservers(Loss);
                deleteObservers();
                return;
            }

            if (tile.adjacentMines().isEmpty())
                adjacent(tile).forEach(Tile::reveal);

            boolean allEmptyRevealed = ❸ Stream.of(tiles)
                .allMatch(t → t.isMine || t.visibility() == Revealed);

            boolean allMinesFlagged = ❸ Stream.of(tiles)
                .allMatch(t → !(t.isMine ^ t.visibility() == Flagged));

            if (allEmptyRevealed || allMinesFlagged) {
                notifyObservers(Victory);
                deleteObservers();
            }
        }
    }

    notifyObservers(tile);
}
```

Le versioni più recenti di Java hanno introdotto la sintassi o `instanceof Tile tile ❶` per il casting:

- se `Object` o non è un'istanza di `Tile` ritorna `false`
- se `Object` o è un'istanza di `Tile` ritorna `true` e genera `Tile tile = (Tile)o`

In questo esempio si può vedere anche l'implementazione dei vincoli `[Constraint.Game.loss_condition]` ❷ e `[Constraint.Game.victory_condition]` con gli `Stream` ❸.

ⓘ Nota

La `switch` expression permette anche di fare casting per casi in cui si devono gestire più tipi.

Controller

Il **Controller**, in questa scelta d'implementazione, ha il compito di **definire gli eventi** ❷ generati dalla View, specificando le interazioni ❸ con il Model e assegnando gli Observer ❶. Inoltre, è il Controller a gestire il timer ❹.

```
public class Controller {
    private Optional<ScheduledFuture<?>> timer;
    private ScheduledExecutorService scheduler;
    private Optional<Game> game;

    public Controller(Minesweeper model, View view) {
        scheduler = Executors.newScheduledThreadPool(1);
        ❶ model.addObserver(view.menu());
        model.load();

        view.menu().play().addActionListener(❷ e -> {
            Game game = new Game();

            ❶ game.addObserver(model);
            ❶ game.addObserver(view.play());
            ❶ game.addObserver(view.play().canvas());
            ❸ game.start();

            this.game = Optional.of(game);
            ❹ timer = Optional.of(
                scheduler.scheduleAtFixedRate(
                    () -> x3 game.update(), 1, 1, TimeUnit.SECONDS));
        });

        view.play().end().addActionListener(❷ e -> {
            ❹ timer.ifPresent(t -> t.cancel(true));
            ❸ game.ifPresent(Game::end);
        });

        view.play().canvas().addMouseListener(
            ❷ new MouseAdapter() {
                @Override
                public void mouseClicked(MouseEvent e) {
                    game.ifPresent(game -> {
                        Canvas canvas = view.play().canvas();

                        int x = (e.getX() - canvas.getWidth() / 2 +
                            5 * Canvas.SCALE) / 30;
                        int y = (e.getY() - canvas.getHeight() / 2 +
                            5 * Canvas.SCALE) / 30;

                        switch (e.getButton()) {
                            case MouseEvent.BUTTON1 ->
                                ❸ game.tiles[y * 10 + x].reveal();
                            case MouseEvent.BUTTON3 ->
                                ❸ game.tiles[y * 10 + x].flag();
                            default -> {}
                        }
                    });
                }
            }
        );
    }
}
```

```

        }
    });
}
}

```

◉ Nota

Esistono altre strategie per implementare il Controller, ma questo è una delle più semplici. Una **strategia più comoda** potrebbe essere quella di definire un'**interfaccia** per elencare le possibili interazioni che il Controller deve definire, e fare in modo che la View implementi tale interfaccia.

View

La parte di **View** è stata largamente discussa nella prima parte di questa guida. Il lettore è invitato a [consultare il codice del progetto](#) per eventuali dubbi.

In conclusione su MVC

L'obiettivo di questa sezione è stato quello di fornire un semplice esempio di progetto realizzato con il pattern architetturale MVC, e alcuni consigli pratici e trucchi per scrivere il codice del Model.

Gli strumenti forniti fino a questo punto dovrebbero bastare per **lavorare serenamente al progetto**. Il capitolo successivo serve per chi vuole chiudere il progetto con la «*ciliegina sulla torta*» (o anche solo per curiosità) usando gli strumenti che si usano tipicamente in **produzione** nei progetti reali.

Integrare il progetto con GitHub

Prima di procedere con l'utilizzo di **GitHub** è fondamentale aver compreso bene il funzionamento di [git](#) ([guida scritta per il corso](#)). GitHub è un forge per repository git che offre una vasta serie di funzionalità per supportare gli sviluppatori.

◉ Nota

Prima di procedere con questa parte è fondamentale aver capito la differenza fra git e GitHub e come vengono usati: trovate tutte le informazioni utili in [questa guida](#).

Usare una project manager (Maven)

Gestire un progetto Java crudo è abbastanza complicato: si presentano tutta una serie di problemi:

- se si vuole aggiungere una libreria bisogna scaricarla e configurarla manualmente
- la struttura del progetto dipende dall'editor, tanto che passare il progetto su un altro editor o su una versione vecchia dello stesso editor è estremamente difficile (e richiede molta configurazione manuale)
- è più complicato eseguire il progetto su un server (Ex. GitHub Action) se si è legati ad un'editor
- serve un risultato consistente
- serve poter automatizzare alcune azioni / poter usare script etc...

Uno dei package manager più famosi per Java (e dei più semplici da usare) è **Maven** (ce ne sono altri come Gradle etc... ad esempio, per progetti Android bisogna usare Gradle etc...)

Maven può essere usato in tutti gli editor: VSCode, Eclipse, IntelliJ, anche da terminale (è molto comodo da terminale, personalmente lo uso con Neovim)! E risolve tutti i problemi sopra elencati.

◉ Nota

per ogni linguaggio di programmazione «mainstream» esiste un qualche tipo di project manager / package manager:

- [npm](#) per JavaScript
 - [cargo](#) per Rust
 - [pip](#) per Python
 - etc...
- come scaricare Maven
 - Windows: <https://maven.apache.org/download.cgi>
 - Ubuntu/Debian: `sudo apt-get install maven`
 - come creare un progetto maven

```
mvn archetype:generate -DgroupId=<package-principale> -DartifactId=<nome-del-progetto> -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

- sostituite al posto di `<package-principale>` il nome del package principale del progetto

- sostituite al posto di `<nome-del-progetto>` il nome della cartella in cui generare il progetto
- `-DarchetypeArtifactId=maven-archetype-quickstart` serve a specificare quale «template» usare per inizializzare il progetto
- `-DinteractiveMode=false` disabilita il prompt interattivo per le altre impostazioni (mettetelo `=true` se volete vedere le altre impostazioni)

ora si può scrivere il codice

Molto facile da usare dai vari editor, ci sono comandi come

- `mvn install`
- `mvn exec:java`
- `mvn javadoc:javadoc`
- `mvn test`
- `mvn clean`

GitHub Pages

[GitHub Pages](#) è uno strumento che permette di pubblicare (gratuitamente) siti web **statici** tramite GitHub (come può essere il caso di **javadoc**).

Releases

Le [Releases](#) permettono di pubblicare file scaricabili: eseguibili di programmi, documenti, asset etc...

GitHub Actions

Le [GitHub Action](#) sono uno strumento che permette di **automatizzare** tutti i processi legati alla produzione del software: testing, generazione di documentazione, generazione di eseguibili, deploy su server, pubblicazione di librerie etc...

Le GitHub Action si possono usare insieme a GitHub Pages per **generare la documentazione** del progetto e **pubblicarla online**.

Si possono usare anche insieme a GitHub Releases per **generare l'eseguibile** del progetto e pubblicarne la versione scaricabile.

Di seguito si vedono due GitHub Action che automatizzano proprio queste due funzionalità.

Anatomia di una GitHub Action

Per creare un'Action basta creare una cartella `.github/workflows/` all'interno del progetto e aggiungervi un file con estensione `.yml` ([YAML](#)) in cui descrivere i compiti dell'Action (ad esempio `progetto/.github/workflows/javadoc.yml`).

```
name: Publish Docs ❶

on: [push ❷, workflow_dispatch x3]

permissions: ❹
  contents: read
  pages: write
  id-token: write

jobs: ❺
  build:
    runs-on: ubuntu-latest ❻
    steps:
      - uses: actions/checkout@v4 ❼

      # ... steps ... ❸

      - run: mvn install javadoc:javadoc ❾

      # ... steps ... ❸
```

Analisi della GitHub Action:

- ❶ imposta il nome dell'Action
- `on`: serve a specificare quando deve essere eseguita l'Action, in particolare:
 - ❷ specifica che l'Action va eseguita quando c'è un `push` nel repository
 - ❸ permette di invocare l'Action manualmente (dal sito di GitHub, nella sezione «Actions» del repository)
- se l'Action deve eseguire alcune azioni particolari (ad esempio pubblicare su GitHub Pages) ha bisogno dei permessi ❹ per poterlo fare (permessi che bisogna specificare esplicitamente per motivi di sicurezza)
- ❺ l'Action deve eseguire uno o più compiti in sequenza
- per poter essere eseguita, l'Action ha bisogno di una macchina virtuale su cui girare, e bisogna specificarne il Sistema Operativo ❻
- spesso si usano «ricette» già pronte ❼, ad esempio, `actions/checkout` carica il repository all'interno della VM per poterci lavorare sopra
- ❸ naturalmente, nei vari step si possono usare anche i comandi messi a disposizione dal Sistema Operativo scelto per far girare l'Action:
 - si possono anche scaricare pacchetti
 - la macchina ha accesso a internet
- in particolare, per generare la documentazione del progetto, avendo usato Maven, basta eseguire i comandi `mvn install` e `mvn javadoc:javadoc` ❾

Generare e pubblicare la documentazione in automatico

File `.github/workflows/javadoc.yml`

ⓘ Nota

Nel progetto [Minesweeper](#) sono presenti entrambe le GitHub Action discusse in questa sezione

Oltre ai comandi del Sistema Operativo (che si possono invocare con `-run: comando`) è possibile usare delle ricette «già pronte» e configurabili, ad esempio: `actions/checkout`, `actions/setup-java`, `actions/configure-pages`, `actions/upload-pages-artifact`, `actions/deploy-pages` etc...

Queste «ricette» semplificano il lavoro di configurare gli strumenti necessari e permettono di interagire con API (come quello di GitHub Pages, per pubblicare la documentazione generata).

```
name: Publish Docs

on: [push, workflow_dispatch]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
      - run: mvn install javadoc:javadoc
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: target/reports/apidocs
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    needs: build
    steps:
      - uses: actions/deploy-pages@v4
```

Generare l'eseguibile in automatico

File .github/workflows/jar.yml

```
name: Build JAR
on: [push, workflow_dispatch]

permissions:
  contents: write
  id-token: write

jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
      - run: mvn clean compile assembly:single
      - name:
        uses: marvinpinto/action-automatic-releases@latest
        with:
          repo_token: ${ secrets.GITHUB_TOKEN }
          automatic_release_tag: 'latest'
          prerelease: false
          files: target/*.jar
      - uses: dev-drprasad/delete-older-releases@v0.3.3
        with:
          keep_latest: 1
          delete_tags: true
          delete_tag_pattern: latest
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

ⓘ Nota

Un altro esempio di GitHub Action è quella usata per generare [questo stesso .pdf](#) :)

Tips & tricks

Compressione di immagini e audio

Cosa fare se lo `.zip` pesa troppo per la consegna

Per i progetti web una delle metriche più importanti è quella della «velocità di caricamento» del sito, avendo tutta una serie di conseguenze sulla performance del sito. Per ottimizzare questa metrica quello che si fa è provare a minimizzare il peso degli asset (immagini, video, audio, etc...) e del codice.

Questo generalmente non avviene per i giochi (ci sono giochi che pesano anche 100GB+). Però, nel contesto specifico del progetto, per poter consegnare, potrebbe essere utile risparmiare un po' di spazio.

Compressione lossy vs loseless

Generalmente esistono due modi di comprimere gli asset:

- **loseless**: la qualità dell'asset non cambia, e si risparmia un po' di spazio
- **lossy**: la qualità dell'asset è inferiore, ma si risparmia molto più spazio

Esempio di compressione

Quanto va ad impattare realmente la compressione? Consideriamo l'esempio di un'immagine 1920×1080 . Per rappresentare ogni pixel sono necessari 4 byte (RGBA), quindi, un'immagine 1920×1080 dovrebbe pesare

- $1920 \times 1080 \times 4 \text{ byte} \approx 8100 \text{ KB}$.



Figura 6: immagine `.png` 1920×1080 pixel

Consideriamo l'immagine in Figura 6: nonostante i nostri calcoli, il formato `.png` con compressione **loseless** la porta a circa 885 KB (un risparmio del 90%!), ma si può fare meglio? Su Windows è possibile ridimensionare l'immagine direttamente dal programma di preview di default (Figura 7), e usando la compressione **lossy** `.jpg`, con perdita

di qualità del 50% si arriva a 417 KB, e spesso la perdita di qualità non si percepisce neanche.

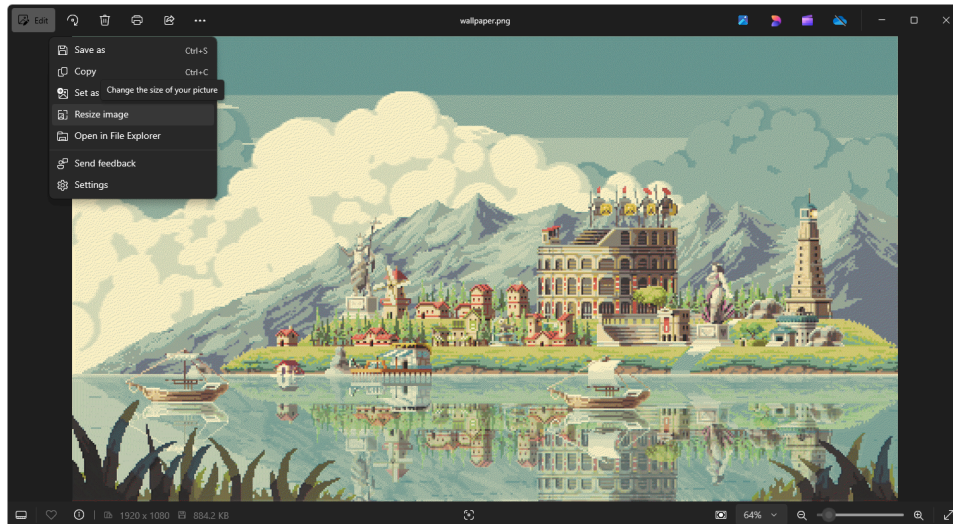


Figura 7: su Windows è possibile ridimensionare e comprimere le immagini

ⓘ Nota

Il peso finale dopo la compressione dipende molto anche dal contenuto dell'immagine: un'immagine 1920×1080 interamente nera si può comprimere in maniera molto più efficace rispetto ad un'immagine ricca di dettagli

Compressione audio

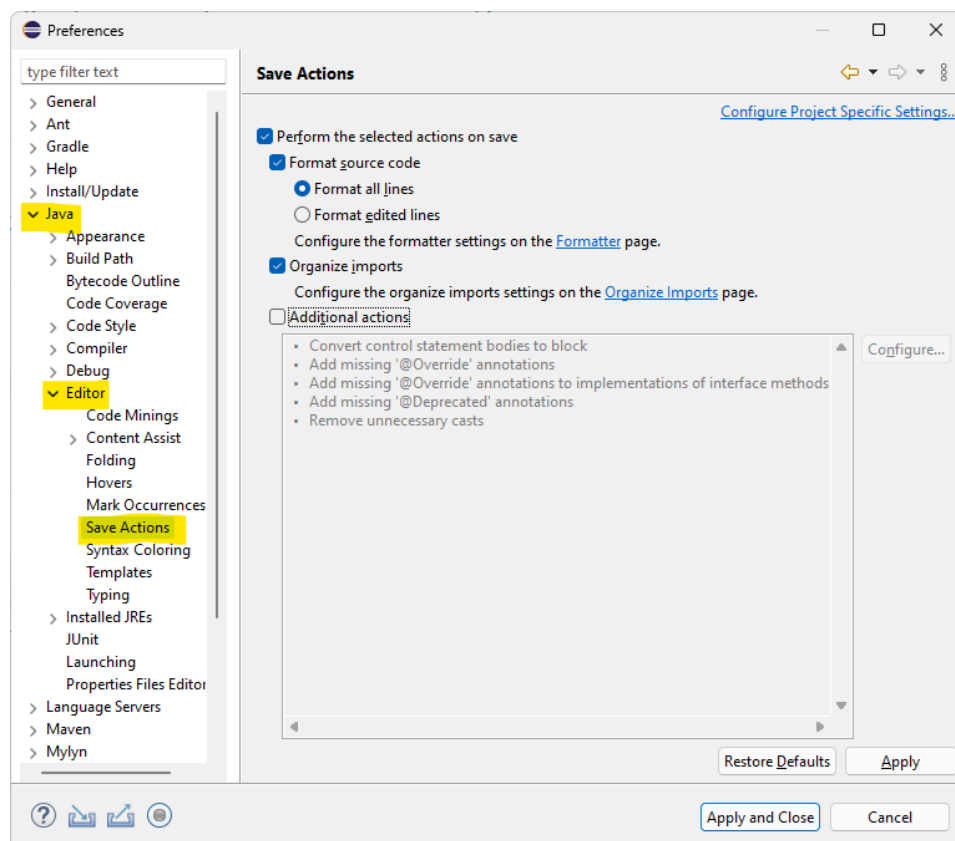
Lo stesso discorso sulla compressione loseless e lossy si può applicare per gli audio (che ho notato essere più problematici): consiglio di usare un formato di compressione lossy come **.mp3**, e di provare a tagliare dove possibile (usando audio più brevi, o di qualità inferiore).

LSP + Formattazione automatica del codice

Nel 2016 la Microsoft ha sviluppato il protocollo [LSP](#) (Language Server Protocol), che ha permesso di sviluppare i Language Server: programmi che offrono funzionalità di **supporto per lo sviluppo di codice**: suggerimenti, autocompletamento, formattazione del codice, diagnostica, linting etc...

La maggior parte degli **editor moderni** (fra cui anche Eclipse) non fanno altro che offrire un'interfaccia grafica per i Language Server (con shortcut, syntax highlighting, configurazione etc...).

Uno dei Language Server più usati per Java è proprio quello di [Eclipse](#). Una delle funzionalità più importanti che offre è la **formattazione del codice**, che permette di mantenere il codice **consistente e pulito**, specialmente quando si lavora in un team. Eclipse (come tanti altri editor) permette di invocare questa funzionalità automaticamente ogni volta che il codice viene salvato:



Shortcut per gli editor

Nella maggior parte degli editor è possibile commentare velocemente il codice (anche più righe alla volta) usando la shortcut `Ctrl + /`